



WOLLO University

Department of software engineering

Blueprint based exit material for

Fundamental software engineering

Prepared by:

@Neyatibeb

KOMBOLCHA, Ethiopia

2015 E.C

Department of software engineering

❖ Basic principle of software engineering

Software engineering is a discipline that applies engineering principles to the design, development, and maintenance of software. It is a complex and ever-evolving field, but there are a number of basic principles that are essential to good software engineering.

Some of the most important basic principles of software engineering include:

- **Planning and requirements analysis:** Before any code is written, it is essential to carefully plan the software project and understand the requirements of the users. This will help to ensure that the software is developed correctly and meets the needs of the users.
- **Design:** The design phase of software engineering is where the high-level architecture of the software is created. This includes decisions about the software's components, how they will interact, and how the software will be implemented.
- **Coding:** The coding phase is where the software is actually written. This is a complex and challenging task, and it is important to follow good coding practices to ensure that the software is readable, maintainable, and efficient.
- **Testing:** Testing is essential to ensure that the software is free of errors and that it meets the requirements of the users. There are a variety of different testing techniques that can be used, and it is important to choose the right techniques for the specific software project.
- **Deployment and maintenance:** Once the software is developed, it needs to be deployed to users and maintained over time. This includes tasks such as fixing bugs, adding new features, and updating the software to keep it secure.
- **Modularity:** Breaking down the software into smaller, independent, and reusable components or modules. This makes the software easier to understand, test, and maintain.
- **Abstraction:** Hiding the implementation details of a module or component and exposing only the necessary information. This makes the software more flexible and easier to change.
- **Encapsulation:** Wrapping the data and functions of a module or component into a single unit, and providing controlled access to that unit. This helps to protect the data and functions from unauthorized access and modification.
- **DRY principle (Don't Repeat Yourself):** Avoiding duplication of code and data in the software. This makes the software more maintainable and less error-prone.
- **KISS principle (Keep It Simple, Stupid):** Keeping the software design and implementation as simple as possible. This makes the software more understandable, testable, and maintainable.
- **YAGNI (You Ain't Gonna Need It):** Avoiding adding unnecessary features or functionality to the software. This helps to keep the software focused on the essential requirements and makes it more maintainable.
- **SOLID principles:** A set of principles that guide the design of software to make it more maintainable, reusable, and extensible. This includes the Single Responsibility Principle, Open/Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, and Dependency Inversion Principle.
- **Test-driven development:** Writing automated tests before writing the code, and ensuring that the code passes all tests before it is considered complete. This helps to ensure that the software meets the requirements and specifications.
- By following these principles, software engineers can develop software that is more reliable, maintainable, and extensible.
- It's also important to note that these principles are not mutually exclusive, and often work together to improve the overall quality of the software.

Additionally

- Better requirement analysis

Department of software engineering

- Clear vision of project
- Separation concern - gives what the actual user needs and hides unnecessary things.
- Think then act
-

In addition to these basic principles, there are a number of other important concepts in software engineering, such as agile development, version control, and code reviews. These concepts can help to improve the quality of software and make the software development process more efficient.

Following the basic principles of software engineering can help to ensure that software projects are successful. By carefully planning, designing, coding, testing, and deploying software, software engineers can create software that is reliable, efficient, and meets the needs of the users.

Here are some additional principles of software engineering that are worth mentioning:

- Modularity: Software should be designed as a collection of independent modules that can be easily reused and maintained.
- Abstraction: Software should be designed to hide implementation details from users. This makes the software easier to understand and use.
- Efficiency: Software should be designed to be efficient in terms of both time and space.
- Reliability: Software should be designed to be reliable and avoid errors.
- Maintainability: Software should be designed to be easy to maintain and update.

These principles are not exhaustive, but they provide a good foundation for understanding the basic principles of software engineering.

❖ Assess software process and state-of-the-arts software process models

Some of the state-of-the-art software process models:

- Waterfall model: The waterfall model is a linear approach to software development. It is divided into a series of phases, such as requirements, design, coding, testing, and deployment. Each phase must be completed before the next phase can begin.
- Iterative model: The iterative model is an incremental approach to software development. The software is developed in a series of iterations, or cycles. Each iteration includes the requirements, design, coding, testing, and deployment of a portion of the software.
- Agile model: The agile model is a lightweight approach to software development. It emphasizes collaboration and flexibility. Agile projects are typically divided into sprints, or short periods of time. The software is developed in a series of sprints, with each sprint resulting in a working product.
- DevOps: DevOps is a set of practices that combine software development (Dev) and IT operations (Ops). The goal of DevOps is to improve the speed and quality of software delivery.

Each of these models has its own strengths and weaknesses. The waterfall model is a good choice for projects with well-defined requirements. The iterative model is a good choice for projects with changing requirements. The agile model is a good choice for projects that need to be delivered quickly. DevOps is a good choice for projects that need to be delivered reliably and efficiently.

Department of software engineering

In addition to these models, there are a number of other software process models that are used in practice. These include the spiral model, the V-model, and the lean model.

The choice of software process model depends on a number of factors, including the size and complexity of the project, the availability of resources, and the preferences of the project team.

Here are some of the factors to consider when assessing a software process:

- The quality of the software: The software process should produce high-quality software that meets the requirements of the users and stakeholders.
- The efficiency of the process: The software process should be efficient, so that the software can be developed and delivered in a timely manner.
- The flexibility of the process: The software process should be flexible, so that it can be adapted to changes in the requirements or the environment.
- The cost of the process: The software process should be cost-effective, so that the software can be developed and delivered within budget.

The assessment of a software process should be conducted by a team of experts who are familiar with the software process models and the factors that contribute to the quality of the software. The assessment should be documented so that the results can be used to improve the software process.

There are four basic software processes

1. **Software specification** (or requirements engineering): Define the main functionalities of the software and the constraints around them.
2. **Software design and implementation**: The software is to be designed and programmed.
3. **Software verification and validation**: The software must conform to its specification and meet the customer's needs.
4. **Software evolution** (software maintenance): The software is being modified to meet customer and market requirements changes.

Phase or step of software process model (SDLC)

1. **Planning and requirement analysis**
 - Planning for the quality assurance requirements and identifications of the risks associated with the projects is also done at this stage
 -
2. **Defining requirement**
3. **Designing software**
4. **Developing the project**
5. **Testing**
6. **Deployment**
7. **Maintenance**

Software process model

- a. Waterfall model
 - Simple to implement
 - The requirement is simple and remain unchanged during the entire project development
 - Start and end point of each phase is fixed

Department of software engineering

- Release date and final cost can be determined before development.
- Not suitable for complex project
- Testing done at a later stage so the risk reduction strategy is difficult to prepare.
- Have five phases Requirements analysis and specification, design, implementation, and unit testing, integration and system testing, and operation and maintenance.
- The developer must complete every phase before the next phase begins.
- When this model is suitable is
 - ✓ When the requirements are constant and not changed regularly.
 - ✓ A project is short, The situation is calm
 - ✓ Where the tools and technology used is consistent and is not changing
 - ✓ When resources are well prepared and are available to use.

b. RAD(rapid application development) model

- Linear sequential, use element based construction approach
- Evolutionary process model
- This model is flexible to change
- deploy an advanced version of the existing software in the market
- Increase reusability of feature
- Reduces development time
- Require high risk designer
- All application is not compatible to RAD
- Low technical risk
- Require user involvement
- When this model is suitable is
 - ✓ When the system should need to create the project that modularizes in a short span time (2-3 months).
 - ✓ Large project
 - ✓ When the requirements are well-known.
 - ✓ When the technical risk is limited.
 - ✓ When there's a necessity to make a system, which modularized in 2-3 months of period.
 - ✓ It should be used only if the budget allows the use of automatic code generating tools.

c. Spiral model

- Linear sequential model, the iterative feature of prototyping, software is developed in a series of incremental releases
- Is chosen if the development team has less experience on similar projects.
- Expensive, require skill
- doesn't necessitate defining requirements at the earliest in the lifecycle
- Is **risk-driven process model**
- It implements the potential for rapid development of new versions of the software.
- High amount of risk analysis
- When this model is suitable is
 - ✓ When deliverance is required to be frequent.
 - ✓ When the project is large
 - ✓ When requirements are unclear and complex
 - ✓ When changes may require at any time
 - ✓ Large and high budget projects

d. V model

- Refers to Verification and Validation Model.
- Easy to understand
- Test method before coding
- Save time, avoid downward defect
- Rigid and least feasible
- Not good for complex project
- Software is developed during the implementation stage, so no early prototypes of the software are produced.
- In this, each phase of SDLC must complete before the next phase starts
- It follows a sequential design process same as the waterfall model. Testing of the device is planned in parallel with a corresponding stage of development.
- When this model is suitable is
 - ✓ When the requirement is well defined and not ambiguous.
 - ✓ The V-shaped model should be used for small to medium-sized projects where requirements are clearly defined and fixed.
 - ✓ The V-shaped model should be chosen when sample technical resources are available with essential technical expertise.

e. Incremental model

- Requirements divided into multiple standalone modules of the software development cycle.
- Errors are easy to be recognized.
- Easier to test and debug, more flexible.
- Simple to manage risk because it handled during its iteration.
- The Client gets important functionality early.
- Need for good planning, Total Cost is high.
- Well defined module interfaces are needed.
- When this model is suitable is
 - ✓ When the requirements are superior.
 - ✓ A project has a lengthy development schedule.
 - ✓ When Software team are not very well skilled or trained.
 - ✓ When the customer demands a quick release of the product.
 - ✓ You can develop prioritized requirements first.

f. Agile model

- Based on iterative development. Agile methods break tasks into smaller iterations, or parts do not directly involve long term planning.
- Face-to-Face Communication with clients, Frequent Delivery
- Efficient design and fulfils the business requirement.
- Anytime changes are acceptable.
- It reduces total development time.
- When this model is suitable is

Department of software engineering

- ✓ When frequent changes are required.
- ✓ When a highly qualified and experienced team is available.
- ✓ When a customer is ready to have a meeting with a software team all the time.
- ✓ When project size is small.

Principles of the agile manifesto are:

- Customer satisfaction is a priority.
- Changes in requirements are welcome, if they occur late in the development process.
- Deliver working software on a regular basis.
- Business people and developers must connect on a daily basis through the project.
- Support people who are interested and passionate about developing.
- Face-to-face communication within a development team.
- Working software indicates progress.
- Sustainable development is aided by agile processes.
- A constant focus on technical excellence and smart design improve agility.
- Simplicity, or the art of minimizing the amount of effort that isn't done, is critical.
- Self-organizing teams
- The team reflects on how to become more effective at regular intervals, then selects its behavior accordingly

g. Iterative model

- Testing and debugging during smaller iteration is easy.
- A Parallel development can plan.
- It is easily acceptable to ever-changing needs of the project.
- Risks are identified and resolved during iteration.
- Limited time spent on documentation and extra time on designing
- It is not suitable for smaller projects.
- More Resources may be required.
- deploy an advanced version of the existing software in the market
- Design can be changed again and again because of imperfect requirements.
- Requirement changes can cause over budget.
- Project completion date not confirmed because of changing requirements.
- When this model is suitable is
 - ✓ When requirements are defined clearly and easy to understand.
 - ✓ When the software application is large.
 - ✓ When there is a requirement of changes in future.

h. Big bang model

- In this model, developers do not follow any specific process.
- There is no planning required, Simple Model.
- Few resources required, Easy to manage
- Flexible for developers, there are high risk and uncertainty.
- Not acceptable for a large project.
- If requirements are not clear that can cause very expensive

Department of software engineering

- Development begins with the necessary funds and efforts in the form of inputs. And the result may or may not be as per the customer's requirement, because in this model, even the customer requirements are not defined.
- When this model is suitable is
 - ✓ As we discussed above, this model is required when this project is small like an academic project or a practical project. This method is also used when the size of the developer team is small and when requirements are not defined, and the release date is not confirmed or given by the customer.

i. Prototype model

- doesn't necessitate defining requirements at the earliest in the lifecycle
- Require user involvement
-

❖ Demonstrate various type of requirement

Software requirements can be classified into various types based on their characteristics and the needs they address. The following are the common classifications of software requirements:

1. **Functional requirements:** These requirements describe what the software system should do. They specify the functionality of the system and the actions it should perform. Examples of functional requirements include user authentication, data entry, and report generation.
2. **Non-functional requirements:** These requirements specify the qualities or characteristics of the software system, such as its performance, reliability, scalability, usability, and security. Non-functional requirements are not directly related to the system's functionality but are critical to its success.
3. **User requirements:** These requirements describe what the end-user wants from the software system. User requirements are usually expressed in natural language and are typically gathered through interviews, surveys, or user feedback.
4. **System requirements:** These requirements specify the technical characteristics of the software system, such as its architecture, hardware requirements, software components, and interfaces. System requirements are typically expressed in technical terms and are often used as a basis for system design.
5. **Business requirements:** These requirements describe the business goals and objectives that the software system is expected to achieve. Business requirements are usually expressed in terms of revenue, market share, customer satisfaction, or other business metrics.
6. **Regulatory requirements:** These requirements specify the legal or regulatory standards that the software system must meet. Regulatory requirements may include data privacy, security, accessibility, or other legal compliance requirements.
7. **Interface requirements:** These requirements specify the interactions between the software system and external systems or components, such as databases, web services, or other software applications.

Additional

Some examples of various types of requirements:

- Functional requirements: These requirements describe what the system should do. For example, a functional requirement for a banking software might be "The system must allow users to deposit money into their accounts."

Department of software engineering

- Non-functional requirements: These requirements describe how the system should behave. For example, a non-functional requirement for a banking software might be "The system must be available 24/7."
- Business requirements: These requirements describe the business needs that the system must meet. For example, a business requirement for a banking software might be "The system must help the bank to reduce fraud."
- Usability requirements: These requirements describe how easy the system should be to use. For example, a usability requirement for a banking software might be "The system must be easy to use by people with disabilities."
- Performance requirements: These requirements describe how well the system should perform. For example, a performance requirement for a banking software might be "The system must be able to handle 1000 transactions per second."
- Security requirements: These requirements describe how secure the system should be. For example, a security requirement for a banking software might be "The system must protect customer data from unauthorized access."

These are just a few examples of the many different types of requirements that can be found in a software project. The specific requirements that are needed will vary depending on the specific project.

It is important to document the requirements for a software project in a clear and concise way. This will help to ensure that the project team understands the requirements and that the software is developed to meet the needs of the users and stakeholders.

Here are some tips for writing good requirements:

- Be clear and concise. The requirements should be easy to understand and should not be ambiguous.
- Be specific. The requirements should be specific enough to guide the development of the software.
- Be measurable. The requirements should be measurable so that it can be determined whether or not they have been met.
- Be realistic. The requirements should be realistic and achievable.
- Be traceable. The requirements should be traceable to the business needs that they are intended to meet.

❖ Create a system, component, or process to meet desired needs.

To create a system, component, or process to meet desired needs. Here are the steps involved:

1. Identify the desired needs. What do you want the system, component, or process to do? What are the specific requirements?
2. Design the system, component, or process. This involves creating a blueprint for how the system will work. The design should take into account the desired needs and the constraints of the system.
3. Implement the system, component, or process. This involves coding the system, building the component, or creating the process.
4. Test the system, component, or process. This involves running the system, component, or process to make sure that it meets the desired needs and works as expected.
5. Deploy the system, component, or process. This involves making the system, component, or process available to users.

Department of software engineering

Here are some additional tips for creating a system, component, or process to meet desired needs:

- Start with a clear understanding of the desired needs. What do you want the system, component, or process to do? What are the specific requirements?
- Use a design methodology that is appropriate for the project. There are many different design methodologies, such as waterfall, iterative, and agile. The choice of methodology will depend on the size and complexity of the project, the availability of resources, and the preferences of the project team.
- Use a coding language that is appropriate for the project. There are many different coding languages, such as Java, Python, and C++. The choice of language will depend on the requirements of the system, the skills of the developers, and the preferences of the project team.
- Test the system, component, or process thoroughly. This is essential to ensure that the system meets the desired needs and works as expected.
- Deploy the system, component, or process in a controlled manner. This will help to ensure that the system is not disrupted and that users are able to use it as expected.

Additional

- ➡ Reversed engineering
 - generating analysis and designing documents
- ➡ Re-engineering
- ➡ Forward engineering